Лабораторная работа №1.

Аутентификация пользователей

Тема: Регистрация и вход пользователей в Express-приложении

Цель: Создать модуль аутентификации Internet Shop с регистрацией и входом пользователя, безопасным хранением passwordHash и разделением Route, Controller и Service.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом `express-shop`. В проекте уже используются Express.js, PostgreSQL и Prisma Client.

На этой лабораторной `argon2.hash()` и `argon2.verify()` используются как готовые операции. Подробно хеширование паролей будет изучено на следующем занятии.

> **Безопасность**
> Не сохраняйте пароль пользователя в открытом виде даже в учебном проекте. В таблице users должен храниться только passwordHash.

1. Установите argon2

```PowerShell
npm install argon2
```

2. Обновите модель User

```prisma/schema.prisma
model User {
  id           Int      @id @default(autoincrement())
  name         String
  email        String   @unique
  passwordHash String   @map("password_hash")
  role         String   @default("user")
  orders       Order[]

  @@map("users")
}
```

Поле `email` используется как идентификатор пользователя. Поле `passwordHash` предназначено для результата безопасного password hashing. Поле `role` понадобится в теме авторизации.

3. Создайте миграцию

```PowerShell
```

```
npx prisma@7 migrate dev --name add_user_auth_fields
npx prisma@7 generate
```

4. Создайте passwordService

services/passwordService.js

```
import * as argon2 from "argon2";

export async function hashPassword(password) {
  return argon2.hash(password);
}

export async function verifyPassword(passwordHash, password) {
  return argon2.verify(passwordHash, password);
}
```

> **На этой работе**
>
> Не изменяйте параметры Argon2. Сейчас задача — понять общий Authentication flow. Алгоритм и настройки password hashing будут разобраны отдельно.

5. Создайте Auth Service

services/authService.js

```
import { prisma } from "../config/prisma.js";
import { hashPassword, verifyPassword } from "./passwordService.js";

export async function registerUser({ name, email, password }) {
  const existing = await prisma.user.findUnique({
    where: { email }
  });

  if (existing) {
    throw new Error("Email already registered");
  }

  const passwordHash = await hashPassword(password);

  return prisma.user.create({
    data: {
      name,
      email,
      passwordHash
    },
    select: {
      id: true,
      name: true,
      email: true,
      role: true
    }
  });
}

export async function authenticateUser(email, password) {
```

```
  const user = await prisma.user.findUnique({
    where: { email }
  });

  if (!user) {
    return null;
  }

  const valid = await verifyPassword(
    user.passwordHash,
    password
  );

  if (!valid) {
    return null;
  }

  return {
    id: user.id,
    name: user.name,
    email: user.email,
    role: user.role
  };
}
```

`registerUser()` создаёт новую учётную запись. `authenticateUser()` проверяет существующую. Ни одна функция не возвращает `passwordHash` клиенту.

6. Создайте Auth Controller

controllers/authController.js

```
import {
  registerUser,
  authenticateUser
} from "../services/authService.js";

export async function register(req, res, next) {
  try {
    const user = await registerUser(req.body);
    res.status(201).json({ user });
  } catch (error) {
    next(error);
  }
}

export async function login(req, res, next) {
  try {
    const { email, password } = req.body;
    const user = await authenticateUser(email, password);

    if (!user) {
      return res.status(401).json({
        error: "Invalid email or password"
      });
    }

    res.json({ user });
  } catch (error) {
    next(error);
  }
```

```
}
```

7. Создайте Auth routes

routes/authRoutes.js

```
import { Router } from "express";
import {
  register,
  login
} from "../controllers/authController.js";

const router = Router();

router.post("/register", register);
router.post("/login", login);

export default router;
```

8. Подключите маршруты

app.js

```
import authRoutes from "./routes/authRoutes.js";

app.use("/api/auth", authRoutes);
```

Auth endpoints

Метод	URL	Назначение
POST	/api/auth/register	Создать учётную запись
POST	/api/auth/login	Проверить email и password

9. Проверьте регистрацию

POST /api/auth/register

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Alex",
  "email": "alex@example.com",
  "password": "ExamplePassword123!"
}
```

Пример ответа

```
{
```

```
  "user": {
    "id": 1,
    "name": "Alex",
    "email": "alex@example.com",
    "role": "user"
  }
}
```

Значение `passwordHash` не должно присутствовать в HTTP-ответе.

10. Проверьте данные в PostgreSQL

Проверка users

```
SELECT
  id,
  name,
  email,
  password_hash,
  role
FROM users
ORDER BY id;
```

В `password_hash` должна находиться длинная строка Argon2, а не исходный пароль `ExamplePassword123!`.

11. Проверьте успешный login

POST /api/auth/login

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "alex@example.com",
  "password": "ExamplePassword123!"
}
```

Пример ответа

```
{
  "user": {
    "id": 1,
    "name": "Alex",
    "email": "alex@example.com",
    "role": "user"
  }
}
```

12. Проверьте неверный password

```
POST /api/auth/login
Content-Type: application/json
```

```
{
  "email": "alex@example.com",
  "password": "wrong-password"
}
```

```
{
  "error": "Invalid email or password"
}
```

13. Проверьте неизвестный email

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "unknown@example.com",
  "password": "ExamplePassword123!"
}
```

```
{
  "error": "Invalid email or password"
}
```

Одинаковая внешняя ошибка

Для неизвестного email и неправильного password клиент должен получать одинаковое сообщение. Не раскрывайте существование учётной записи через login endpoint.

14. Проверьте повторную регистрацию

Повторите `POST /api/auth/register` с уже существующим email. PostgreSQL и Service не должны позволить создать вторую учётную запись с тем же email.

15. Подключите frontend

Пример login на frontend

```
const response = await fetch("http://localhost:3000/api/auth/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ email, password })
});

const data = await response.json();

if (response.ok) {
  console.log("Authenticated user:", data.user);
} else {
  console.error(data.error);
```

```
}
```

> **Пока без session или JWT**
> На этой лабораторной login только проверяет credentials. Механизм сохранения аутентифицированного состояния будет добавлен на следующих занятиях.

Самостоятельное задание

Задание 1. Добавьте проверку данных при регистрации
При регистрации пользователя через POST /api/auth/register сервер должен проверять полученные данные до сохранения в PostgreSQL.
Требования к проверке:

- name — обязательная непустая строка.
- email — обязательная строка, содержащая адрес электронной почты корректного формата.
- password — обязательная строка длиной не менее 8 символов.
- email не должен повторять адрес уже зарегистрированного пользователя.
- Если данные некорректны, сервер должен вернуть HTTP-статус 400 Bad Request и сообщение, указывающее на причину ошибки.
- Если email уже зарегистрирован, сервер должен вернуть 409 Conflict.

Пример некорректного запроса:

```
{
  "name": "",
  "email": "alex@example.com",
  "password": "123"
}
```

Ожидаемый результат: пользователь не создаётся, сервер возвращает ошибку проверки данных.

Что необходимо сделать:

- Создать файл middleware/validateAuth.js с функцией проверки данных регистрации.
- Подключить middleware к маршруту POST /api/auth/register.
- Проверить регистрацию с пустым именем, неправильным email и коротким паролем.
- Попытаться зарегистрировать двух пользователей с одинаковым email.

Задание 2. Добавьте проверку данных при входе
При входе через POST /api/auth/login необходимо проверить, что клиент передал email и пароль.
Требования:

- Email должен быть указан и иметь корректный формат.
- Пароль должен быть указан.
- При отсутствии обязательных данных сервер возвращает 400 Bad Request.
- При неверном пароле или несуществующем email сервер возвращает 401 Unauthorized.
- Для неверного пароля и несуществующего email должно использоваться одно одинаковое сообщение:
-

```
{
  "error": "Invalid email or password"
}
```

Это необходимо, чтобы по сообщению об ошибке нельзя было определить, существует ли пользователь с указанным email.

Что необходимо сделать:

- Добавить функцию проверки данных входа в middleware/validateAuth.js.
- Подключить её к маршруту POST /api/auth/login.
- Проверить вход с правильными данными, неправильным паролем и несуществующим email.
- Убедиться, что ответы в двух последних случаях одинаковые.

Задание 3. Проверьте безопасность HTTP-ответов
При регистрации и входе сервер не должен возвращать клиенту хеш пароля passwordHash.
Что необходимо сделать:

- Зарегистрировать нового пользователя.
- Проверить JSON-ответ сервера.
- Выполнить вход под этим пользователем.
- Проверить JSON-ответ ещё раз.

В ответе допускаются поля:

```
{
  "user": {
    "id": 1,
    "name": "Alex",
    "email": "alex@example.com",
    "role": "user"
  }
}
```

Полей password и passwordHash быть не должно.
В таблице users, напротив, должен храниться password_hash, содержащий хеш Argon2, а не исходный пароль.

Дополнительное задание. Создайте простую страницу входа.
Создайте HTML-страницу login.html, позволяющую пользователю выполнить вход через браузер.

Требования к странице:

- Поле для ввода email.
- Поле для ввода пароля (type="password").
- Кнопка «Войти».
- Отправка HTTP-запроса POST /api/auth/login с помощью fetch().
- Отображение имени пользователя после успешного входа.
- Отображение сообщения об ошибке при неправильных учётных данных.
-

Пример ожидаемого поведения:
При правильных данных:
«Вход выполнен. Добро пожаловать, Alex!»
При неправильном email или пароле:
«Неверный email или пароль».